2 Reguläre Sprachen und endliche Automaten

Vorlesung: 22.04.26

Wie können wir potentiell unendlich große Mengen von Wörtern darstellen? Eine Lösung für dieses Problem sahen wir bereits im vorherigen Kapitel, als wir die (unendlich große) Menge der binär codierten Primzahlen mit Hilfe der folgenden Sprache darstellten.

$$L_{\mathsf{prim}} = \{w \in \{0,1\}^* \mid w \text{ ist Binärcodierung einer Primzahl}\}$$

Ein weiteres Beispiel ist die folgende Sprache.

$$L_{\mathsf{even}} = \{w \in \{0,1\}^* \mid \text{ die Anzahl der Einsen in } w \text{ ist gerade. }\}$$

Eine häufig interessante Fragestellung für ein gegebenes Wort w und eine Sprache L ist: „Ist w in L enthalten?“ (Also: „Gilt $w \in L$?“) Wir nennen dieses Entscheidungsproblem das *Wortproblem*. Eine konkrete Instanz des Wortproblems wäre z.B. „$1100101 \in L_{\mathsf{prim}}$?“ oder „$1100101 \in L_{\mathsf{even}}$?“

Die obige Darstellung der unendlichen Mengen L_{prim} und L_{even} ist zwar sehr kompakt, wir können daraus aber nicht direkt ein Vorgehen zur Lösung des Wortproblems ableiten. Wir müssen zunächst verstehen, was die Begriffe „Binärcodierung“, „Primzahl“ oder „gerade Anzahl“ bedeuten und für L_{prim} und L_{even} jeweils einen Algorithmus zur Entscheidung entwickeln.

In diesem Kapitel werden wir mit *endlichen Automaten* einen weiteren Formalismus kennenlernen, um (potentiell unendlich große) Mengen von Wörtern darzustellen. Ein Vorteil dieser Darstellung ist, dass es einen einheitlichen und effizienten Algorithmus für das Wortproblem gibt. Wir werden aber auch sehen, dass sich nicht jede Sprache (z.B. L_{prim}) mit Hilfe eines endlichen Automaten darstellen lässt.

2.1 Endliche Automaten

Wir beschreiben zunächst informell die Bestandteile eines endlichen Automaten:

Endliches Band (read-only; jede Zelle enthält ein $a_i \in \Sigma$; der Inhalt des Bands ist das *Eingabewort* bzw. die *Eingabe*)

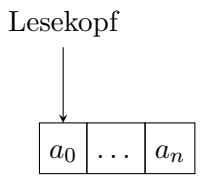


Abb. 1: Endliches Band

Lesekopf

- Der *Lesekopf* zeigt auf ein Feld des Bands, oder hinter das letzte Feld.
- Er bewegt sich feldweise nach rechts; andere Bewegungen (Vor- bzw. Zurückspulen) sind nicht möglich.
- Wenn er hinter das letzte Zeichen zeigt, *stoppt* der Automat. Er muss sich nun „entscheiden“ ob er das Wort *akzeptiert* oder nicht.

Zustände Zu jedem Zeitpunkt befindet sich der Automat in einem Zustand q aus einer *endlichen* Zustandsmenge Q.

Startzustand $q^{\mathsf{init}} \in Q$

Akzeptierende Zustände $F \subseteq Q$

Transitionsfunktion Im Zustand q beim Lesen von a gehe in Zustand $q' := \delta(q, a)$.

Der endliche Automat akzeptiert eine Eingabe, falls er in einem akzeptierenden Zustand stoppt.

Bsp. 2.1: Aufgabe:
„Erkenne alle Stapel von Macarons, in denen höchstens ein grünes Macaron vorkommt.“

[1]

Ein passendes Alphabet wäre $\Sigma = \{\texttt{grün}, \texttt{nicht-grün}\}$. Wir definieren die folgenden Zustände. (Die Metapher hier ist: „wenn ich mehr als ein grünes Macaron esse, wird mir übel, und das wäre nicht akzeptabel“.)

Zustand	Bedeutung
q_0	„alles gut“
q_1	„mir wird schon flau“
q_2	„mir ist übel“

Der Startzustand ist q_0. Akzeptierende Zustände sind q_0 und q_1. Die Transitionsfunktion δ ist durch die folgende Tabelle gegeben.

[1] Von links nach rechts:
By Mariajudit - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=48726001
By Michelle Naherny - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=44361114
By Keven Law - originally posted to Flickr as What's your Colour???, CC BY-SA 2.0, https://commons.wikimedia.org/w/index.php?curid=6851868

	grün	nicht-grün	
q_0	q_1	q_0	wechsle nach q_1 falls grün, ansonsten verweile
q_1	q_2	q_1	wechsle nach q_2 falls grün, ansonsten verweile
q_2	q_2	q_2	verweile, da es nichts mehr zu retten gibt

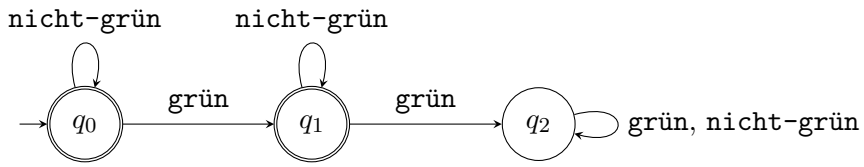
Def. 2.1 (DEA)**:** Ein *deterministischer endlicher Automat (DEA)*[2] ist ein 5-Tupel

$$\mathcal{A} = (\Sigma, Q, \delta, q^{\mathsf{init}}, F).$$

Dabei ist

- Σ ein Alphabet,
- Q eine *endliche* Menge, deren Elemente wir *Zustände* nennen,
- $\delta : Q \times \Sigma \to Q$ eine Funktion, die wir *Transitionsfunktion* nennen,
- $q^{\mathsf{init}} \in Q$ ein Zustand, den wir *Startzustand* nennen und
- $F \subseteq Q$ eine Teilmenge der Zustände, deren Elemente wir *akzeptierende* Zustände nennen. ◇

DEAs lassen sich auch graphisch darstellen. Dabei gibt man für den Automaten einen gerichteten Graphen an. Die Knoten des Graphen sind die Zustände, und mit Zeichen beschriftete Kanten zeigen, welchen Zustandsübergang die Transitionsfunktion für das nächste Zeichen erlaubt. Der Startzustand ist mit einem unbeschrifteten Pfeil markiert, akzeptierende Zustände sind doppelt eingekreist. Hier ist die graphische Darstellung von $A_{\texttt{Macaron}}$ aus Beispiel 2.1:



Die folgenden beiden Definitionen erlauben uns mit Hilfe eines DEA eine Sprache zu charakterisieren. Hierbei sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\mathsf{init}}, F)$ ein DEA.

Def. 2.2: Die *induktive Erweiterung* von $\delta : Q \times \Sigma \to Q$ auf Wörter, $\tilde{\delta} : Q \times \Sigma^* \to Q$, ist (induktiv) definiert durch

1. $\tilde{\delta}(q, \varepsilon) = q$ (Wortende erreicht)

[2] engl. DFA $\hat{=}$ deterministic finite automaton

2. $\tilde{\delta}(q, aw) = \tilde{\delta}(\delta(q, a), w)$ (Rest im Folgezustand verarbeiten) ◇

Def. 2.3: Sei $\mathcal{A} = (\Sigma, Q, \delta, q^{\mathsf{init}}, F)$. Ein Wort $w \in \Sigma^*$ wird von $\mathcal{A}$ *akzeptiert*, falls $\tilde{\delta}(q^{\mathsf{init}}, w) \in F$. Die *von $\mathcal{A}$ akzeptierte Sprache*, geschrieben $L(\mathcal{A})$, ist die Menge aller Wörter, die von $\mathcal{A}$ akzeptiert werden. D.h.,

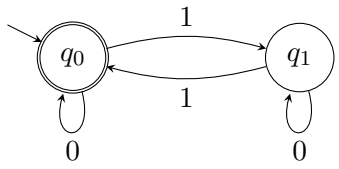
$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \tilde{\delta}(q^{\mathsf{init}}, w) \in F\}.$$

Eine durch einen DEA akzeptierte Sprache heißt *regulär*. ◇

Bsp. 2.2: Ein möglicher DEA für die Sprache

$$L_{\mathsf{even}} = \{w \in \{0,1\}^* \mid \text{ Die Anzahl der Einsen in } w \text{ ist gerade. }\}$$

aus der Einleitung dieses Kapitels hat die folgende graphische Repräsentation.



Frage: Angenommen wir haben zwei DEAs A_1 und A_2, können wir dann immer einen DEA konstruieren der den Schnitt $L(A_1) \cap L(A_2)$ akzeptiert?

Idee: Lasse beide DEAs parallel laufen, akzeptiere nur wenn beide DEAs akzeptieren.

Frage: Lassen sich zwei *parallel laufende* DEAs in einem einzigen DEA implementieren?

Die nächste Konstruktion und der drauf folgende Satz liefern eine positive Antwort auf diese Frage.

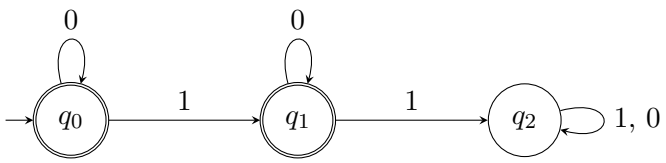
Def. 2.4 (Produktautomat für Schnitt)**:** Seien $A_1 = (\Sigma, Q_1, \delta_1, q_1^{\mathsf{init}}, F_1)$ und $A_2 = (\Sigma, Q_2, \delta_2, q_2^{\mathsf{init}}, F_2)$ DEAs. Wir definieren den *Produktautomaten für Schnitt* als den DEA $A_\cap = (\Sigma, Q_\cap, \delta_\cap, q_\cap^{\mathsf{init}}, F_\cap)$ mit den folgenden Komponenten.

$$\begin{aligned} Q_\cap &= Q_1 \times Q_2 \\ \delta_\cap((q_1, q_2), a) &= (\delta_1(q_1, a), \delta_2(q_2, a)) \ , \quad \text{für alle } a \in \Sigma \\ q_\cap^{\mathsf{init}} &= (q_1^{\mathsf{init}}, q_2^{\mathsf{init}}) \\ F_\cap &= F_1 \times F_2 \end{aligned}$$

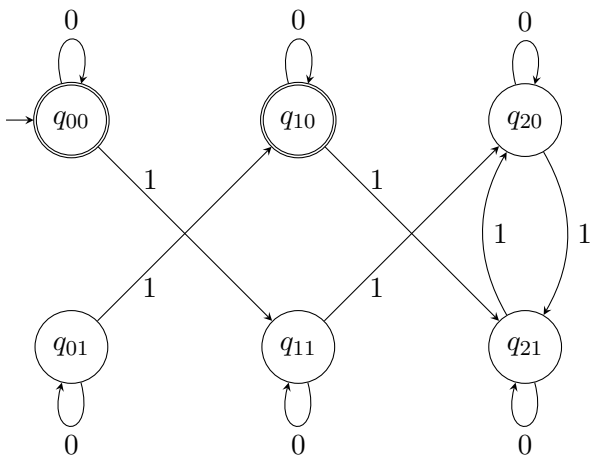
◇

Vorlesung: 28.04.26

Im folgenden Beispiel sei A_1 der DEA über dem Alphabet $\Sigma = \{0, 1\}$, dessen graphische Repräsentation nahezu mit $A_{\texttt{Macaron}}$ identisch ist.



Bsp. 2.3: Der Produktautomat für Schnitt von A_1 und A_{even} hat die folgende graphische Repräsentation, wobei wir um Platz zu sparen „q_{ij}“ statt „(q_i, q_j)“ schreiben.



Satz 2.1: Für beliebige DEAs A_1 und A_2 akzeptiert der entsprechende Produktautomat für Schnitt die Sprache $L(A_1) \cap L(A_1)$.

Beweis: [3] Seien $A_1 = (\Sigma, Q_1, \delta_1, q_1^{\mathsf{init}}, F_1)$ und $A_2 = (\Sigma, Q_2, \delta_2, q_2^{\mathsf{init}}, F_2)$ DEAs und $A_\cap = (\Sigma, Q_\cap, \delta_\cap, q_\cap^{\mathsf{init}}, F_\cap)$ der zugehörige Produktautomat für Schnitt. Wir zeigen zunächst per Induktion über w, dass für alle $w \in \Sigma^*$, für alle $q_1 \in Q_1$ und für alle $q_2 \in Q_2$ die folgende Gleichung $\varphi(w)$ gilt.

$$\varphi(w) := \tilde{\delta}_\cap((q_1, q_2), w) = (\tilde{\delta}_1(q_1, w), \tilde{\delta}_2(q_2, w))$$

[3] Dieser erste Beweis ist außergewöhnlich detailliert. In den folgenden Beweisen werden wir einfache Umformungen zusammenfassen.